WEEK-8

1. TITLE OF THE EXPERIMENT

Student Registration CRUD Application Using MERN Stack

2. AIM

To develop a Student Registration web application using the MERN stack that allows users to register student details and perform Create, Read, Update, and Delete (CRUD) operations using MongoDB. MERN_Student_Registration_Lab

3. PROCEDURE

1. Create a main folder named MERN_MINI_PROJ.
2. Create frontend and backend folders inside the project folder.
3. Create the React frontend using Vite.
4. Install the required frontend dependencies.
5. Initialize the Node.js backend using npm init.
6. Install Express.js, Mongoose and CORS packages.
7. Connect the backend to MongoDB using Mongoose.
8. Create the studentdb database and students collection.
9. Develop the student registration form using React.
10. Use useState() to handle form data.
11. Use Fetch API to communicate with the Express server.
12. Implement GET, POST, PUT and DELETE REST APIs.
13. Display registered students in a table.
14. Add Edit and Delete buttons for each student record.
15. Run both frontend and backend servers and test all CRUD operations.

5.CODE

Server.js

```
import express from "express";
import mongoose from "mongoose";
import cors from "cors";
const app = express();
app.use(cors());
app.use(express.json());
// MongoDB connection
mongoose
  .connect("mongodb://127.0.0.1:27017/studentdb")
  .then(() => {
    console.log("MongoDB connected");
  })
  .catch((error) => {
    console.log("MongoDB connection error:", error.message);
  });
// Schema means structure of Data to be stored in database
const studentSchema = new mongoose.Schema({
  name: String,
  email: String,
  password: String,
  gender: String,
  country: String,
  languages: [String]
});
// Model
const Student = mongoose.model("Student", studentSchema);
// Test route
app.get("/", (req, res) => {
  res.send("Express server is working");
});
```

```
// READ
app.get("/students", async (req, res) => {
  const students = await Student.find();
  res.json(students);
});
// CREATE
app.post("/students", async (req, res) => {
  const student = new Student(req.body);
  await student.save();
  res.json({
    message: "Registration successful"
  });
});
// UPDATE
app.put("/students/:id", async (req, res) => {
  await Student.findByIdAndUpdate(
    req.params.id,
    req.body
  );
  res.json({
    message: "Student updated successfully"
  });
});
// DELETE
app.delete("/students/:id", async (req, res) => {
  await Student.findByIdAndDelete(req.params.id);
  res.json({
    message: "Student deleted successfully"
  });
});
// Start server
app.listen(5000, () => {
```

```
  console.log("Server running on port 5000");
});
```

B. Frontend – App.jsx

```
import heroImg from './assets/hero.png'
import reactLogo from './assets/react.svg'
import viteLogo from './assets/vite.svg'
import './App.css'
import { useEffect, useState } from "react";
const API = "http://localhost:5000";
funtion App() {
  const emptyForm = {
    name: "",
    email: "",
    password: "",
    gender: "",
    country: "",
    languages: []
  };
  const [form, setForm] = useState(emptyForm);
  const [students, setStudents] = useState([]);
  const [message, setMessage] = useState("");
  const [editId, setEditId] = useState("");
  // Get students when page opens
  useEffect(() => {
    loadStudents();
  }, []);
  // READ
  function loadStudents() {
    fetch(API + "/students")
      .then(response => response.json())
      .then(data => {
        setStudents(data);
      })
```

```
    .catch(error => {
      console.log(error);
      setMessage("Start the Express server first");
    });
}
// Text, email, password, radio and select
function handleChange(event) {
  const { name, value } = event.target
  setForm({
    ...form,
    [name]: value
  });
}
// Checkbox
function handleLanguage(event) {
  const value = event.target.value;
  const checked = event.target.checked;
  let languages = form.languages;
  if (checked)
    languages = [...languages, value];
  } else {
    languages = languages.filter(
      item => item !== value
    );
  }
  setForm({
    ...form,
    languages: languages
  });
}
// CREATE or UPDATE
function saveStudent(event) {
  event.preventDefault()
```

```
  let url = API + "/students";
  let method = "POST";
  if (editId !== "") {
    url = API + "/students/" + editId;
    method = "PUT";
  }
  fetch(url, {
    method: method,
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify(form)
  })
    .then(response => response.json())
    .then(data => {
      setMesage(data.message);
      setForm(emptyForm);
      setEditId("");
      loadStudents();
    })
    .catch(error => {
      console.log(error);
      setMessage("Could not connect to Express");
    });
}
// EDIT
function editStudent(student) {
  setForm({
    name: student.name,
    email: student.email,
    password: student.password,
    gender: student.gender,
    country: student.country,
```

```
    languages: student.languages
  });
  setEditId(student._id);
  setMessage("Edit the data and click Update");
}
// DELETE
function deleteStudent(id) {
  fetch(API + "/students/" + id, {
    method: "DELETE"
  })
    .then(response => response.json())
    .then(data => {
      setMessage(data.message)
      loadStudents();
    });
}
// RESET
function resetForm() {
  setForm(emptyForm);
  setEditId("");
  setMessage("");
}
return (
  <div className="page">
    <div className="form-box">
      <h1>Student Registration</h1>
      <form onSubmit={saveStudent}>
        <label>Name</label>
        <input
          type="text"
          name="name"
          value={form.name}
          onChange={handleChange}
```

```
  required
/>
<label>Email</label>
<
/>
<label>Password</label>
<input
  type="password"
  name="password"
  value={form.password}
  onChange={handleChange}
  required
/
<label>Gender</label
<div className="row">
  <label>
    <input
      type="radio"
      name="gender"
      value="Male"
      checked={form.gender === "Male"}
      onChange={handleChange}
    />
    Male
  </label>
  <label>
    <input
      type="radio"
      name="gender"
      value="Female"
      checked={form.gender === "Female"}
      onChange={handleChange}
    />
```

```
    Female
  </label>
</div>
<label>Country</label>
<select
  name="country"
  value={form.country}
  onChange={handleChange}
  required
>
  <option value="">
    Select Country
  </option>
  <option value="India">
    India
  </option>
  <option value="USA">
    USA
  </option>
  <option value="UK">
    UK
  </option>
  <option value="Australia">
    Australia
  </option>
</select>
<label>Languages</label>
<div className="row">
  <label>
    <input
      type="checkbox"
      value="English"
      checked={form.languages.includes("English")}
```

```
          onChange={handleLanguage}
        />
        English
      </label>
      <label>
        <input
          type="checkbox"
          value="Hindi"
          checked={form.languages.includes("Hindi")}
          onChange={handleLanguage}
        />
        Hindi
      </label>
      <label>
        <input
          type="checkbox"
          value="Telugu"
          checked={form.languages.includes("Telugu")}
          onChange={handleLanguage}
        />
        Telugu
      </label>
    </div>
    <button type="submit">
      {editId === "" ? "Register" : "Update"}
    </button>
    <button
      type="button"
      onClick={resetForm}
    >
      Reset
    </button>
  </form>
```

```
  {message && (
    <div className="message">
      {message}
    </div>
  )}
</div>
<div className="data-box">
  <h2>Rgistered Students</h2>
  <table>
    <thead>
      <tr>
        <th>Name</th>
        <th>Email</th>
        <th>Gender</th>
        <th>Country</th>
        <th>Languages</th>
        <th>Actions</th>
      </tr>
    </thead>
    <tbody>
      {students.map(student => (
        <tr key={student._id}>
          <td>{student.name}</td>
          <td>{student.email}</td>
          <td>{student.gender}</td>
          <td>{student.country}</td>
          <td>
            {student.languages.join(", ")}
          </td>
          <td>
            <button
              onClick={() => editStudent(student)}
            >
```

```
                Edit
              </button>
              <button
                onClick={() =>
                  deleteStudent(student._id)
                }
              >
                Delete
              </button>
            </td>
          </tr>
        ))}
      </tbody>
    </table>
  </div>
 </div>
 );
}
export default App;
```

3.Package.json

```
{
 "name": "mern-registration-backend",
 "version": "1.0.0",
 "type": "module",
 "scripts": {
  "start": "node server.js"
 },
 "dependencies": {
  "cors": "^2.8.5",
  "express": "^5.1.0",
  "mongoose": "^8.18.0"
 }
}
```

5. COMMANDS TO RUN

Create Frontend

npm create vite@latest frontend -- --template react

cd frontend

npm install

Create Backend

cd ..

mkdir backend

cd backend

npm init -y

Run Backend

node server.js

Backend URL:

http://localhost:5000

Run Frontend

Open a new terminal:

cd frontend

npm run dev

Frontend URL:

http://localhost:5173

6. TECHNOLOGIES USED

Frontend : React.js

Backend : Node.js

Server : Express.js

Database : MongoDB

ODM : Mongoose

Communication : Fetch API

Styling : CSS

Build Tool : Vite

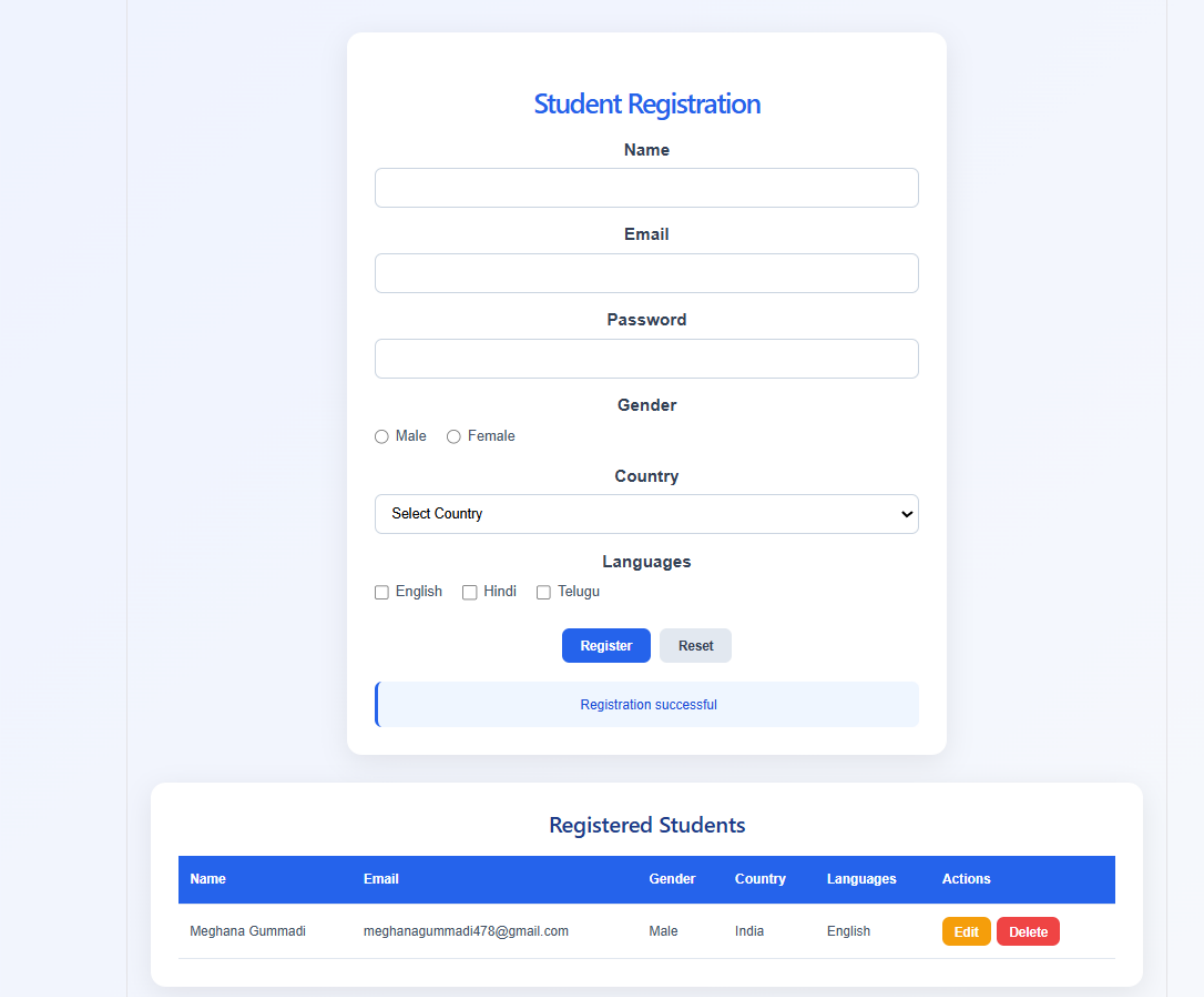
Editor : Visual Studio Code

7. FOLDER STRUCTURE

```
WEEK-10/
│
└── MERN_MINI_PROJ/
    │
    ├── backend/
    │   ├── node_modules/
    │   ├── package.json
    │   ├── package-lock.json
    │   └── server.js
    │
    ├── frontend/
    │   ├── node_modules/
    │   ├── public/
    │   ├── src/
    │   │   ├── assets/
    │   │   ├── App.css
    │   │   ├── App.jsx
    │   │   ├── index.css
    │   │   └── main.jsx
    │   │
    │   ├── .gitignore
```

```
│   ├── eslint.config.js
│   ├── index.html
│   ├── package.json
│   ├── package-lock.json
│   ├── README.md
│   └── vite.config.js
│
└── .gitignore
```

8. OUTPUT



9. RESULT

Thus, the Student Registration CRUD Application was successfully developed using the MERN stack, and the Create, Read, Update, and Delete operations were successfully performed using MongoDB.